

.NET Core Fundamentals Report

1. .NET Versions Evolution

The .NET ecosystem originated with **.NET Framework**, a Windows-exclusive runtime that powered traditional desktop and web applications for over a decade. To meet modern cloud and cross-platform demands, Microsoft introduced **.NET Core**, a redesigned, high-performance, open-source framework. Today, Microsoft has unified these branches under a single platform beginning with .NET 5 and continuing through modern Long-Term Support (LTS) releases like **.NET 8** and **.NET 9**, offering a streamlined runtime for cloud, web, mobile, and IoT development.

2. Namespaces

A **Namespace** serves as a logical container used to organize code structures such as classes, interfaces, and enums, while effectively preventing naming collisions across large codebases. By grouping related capabilities under distinct names (e.g., `System.Text` or `System.IO`), developers can cleanly segment project architecture, control access scope, and easily import required features using the `using` directive.

3. .NET Core Architecture & Advantages

.NET Core is a lightweight, cross-platform framework designed to run seamlessly across Windows, Linux, and macOS environments. Built specifically for modern microservices and containerized deployments (like Docker), it features an ultra-fast execution engine, modular NuGet package management, and side-by-side version execution, making it the industry standard for scalable backend development.

4. Solution File (.sln)

A **Solution** (.sln) is a high-level administrative container in .NET environment that ties together one or more related projects (.csproj). It maintains build configurations, dependency mappings, source control bindings, and project relationships, allowing developers to manage multi-layered architectures—such as API, Application Core, and Infrastructure layers—within a unified workspace.

Bonus Task

1. Reducing JIT Overhead Just-In-Time (JIT) compilation converts Intermediate Language (IL) into machine code dynamically at runtime, which can introduce startup latency and performance overhead. To mitigate this, .NET provides several optimizations:

- **Native AOT (Ahead-Of-Time):** Compiles C# IL directly into architecture-specific machine code during the build process, eliminating JIT overhead, speeding up startup time, and reducing memory usage.
- **Tiered Compilation:** Executes code initially using a quick, unoptimized JIT pass for instant startup, then re-compiles frequently executed ("hot") code paths with deep optimizations in the background.
- **ReadyToRun (R2R):** Embeds pre-compiled native code alongside standard IL in the binaries to speed up cold-start times while preserving cross-platform capabilities.